

AFRILANG Stdlib

std/c/nlpoverlap

Français. Bibliothèque AFRILANG 'std/c/nlpoverlap' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : overLen, overUpper, overLower, overTrim, overSplit, overJoin, overReplace.

```
'import "std/c/nlpoverlap"' · 'use nlpoverlap'
```

```
- 'overLen(s text) → number' - 'overUpper(s text) → text' - 'overLower(s text) → text' - 'overTrim(s text) → text' - 'overSplit(s text, delim text) → list text' - 'overJoin(parts list text, delim text) → text' - 'overReplace(s text, from text, to text) → text'
```

English. AFRILANG library 'std/c/nlpoverlap' (tier complex, category complex). Exports 7 function(s): overLen, overUpper, overLower, overTrim, overSplit, overJoin, overReplace.

```
'import "std/c/nlpoverlap"' · 'use nlpoverlap'
```

```
- 'overLen(s text) → number' - 'overUpper(s text) → text' - 'overLower(s text) → text' - 'overTrim(s text) → text' - 'overSplit(s text, delim text) → list text' - 'overJoin(parts list text, delim text) → text' - 'overReplace(s text, from text, to text) → text'
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/nlpooverlap' (niveau complexe, catégorie complexe). Elle expose 7 fonction(s) : overLen, overUpper, overLower, overTrim, overSplit, overJoin, overReplace.

'import "std/c/nlpooverlap"' · 'use nlpooverlap'

```
- `overLen(s text) → number`
- `overUpper(s text) → text`
- `overLower(s text) → text`
- `overTrim(s text) → text`
- `overSplit(s text, delim text) → list text`
- `overJoin(parts list text, delim text) → text`
- `overReplace(s text, from text, to text) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/nlpooverlap"
use nlpooverlap
```

Niveau : complexe · Catégorie : Modules complexe (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- overLen(s text) → number•
- overUpper(s text) → text•
- overLower(s text) → text•
- overTrim(s text) → text•
- overSplit(s text, delim text) → list•
- overJoin(parts list text, delim text) → text•
- overReplace(s text, from text, to text) → text

4. Exemples d'utilisation

```
import "std/c/nlpooverlap"
use nlpooverlap
```

```
create result = overLen(/* args */)
say result
```

```
create result = overUpper(/* args */)
say result
```

```
create result = overLower(/* args */)
say result
```

```
create result = overTrim(/* args */)
say result
```

```
create result = overSplit(/* args */)
say result
```

```
create result = overJoin(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/nlpoverlap"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module nlpoverlap

```
function overLen(s text) returns number
end
```

```
function overUpper(s text) returns text
end
```

```
function overLower(s text) returns text
end
```

```
function overTrim(s text) returns text
end
```

```
function overSplit(s text, delim text) returns list text
end
```

```
function overJoin(parts list text, delim text) returns text
end
```

```
function overReplace(s text, from text, to text) returns text
end
end
```

English

1. Overview

AFRILANG library 'std/c/nlpooverlap' (tier complex, category complex). Exports 7 function(s): overLen, overUpper, overLower, overTrim, overSplit, overJoin, overReplace.

'import "std/c/nlpooverlap"' · 'use nlpooverlap'

```
- `overLen(s text) → number`
- `overUpper(s text) → text`
- `overLower(s text) → text`
- `overTrim(s text) → text`
- `overSplit(s text, delim text) → list text`
- `overJoin(parts list text, delim text) → text`
- `overReplace(s text, from text, to text) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/nlpooverlap"
use nlpooverlap
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- overLen(s text) → number•
- overUpper(s text) → text•
- overLower(s text) → text•
- overTrim(s text) → text•
- overSplit(s text, delim text) → list•
- overJoin(parts list text, delim text) → text•
- overReplace(s text, from text, to text) → text

4. Usage examples

```
import "std/c/nlpooverlap"
use nlpooverlap
```

```
create result = overLen(/* args */)
say result
```

```
create result = overUpper(/* args */)
say result
```

```
create result = overLower(/* args */)
say result
```

```
create result = overTrim(/* args */)
say result
```

```
create result = overSplit(/* args */)
say result
```

```
create result = overJoin(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/nlpoverlap"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module nlpoverlap

```
function overLen(s text) returns number
end
```

```
function overUpper(s text) returns text
end
```

```
function overLower(s text) returns text
end
```

```
function overTrim(s text) returns text
end
```

```
function overSplit(s text, delim text) returns list text
end
```

```
function overJoin(parts list text, delim text) returns text
end
```

```
function overReplace(s text, from text, to text) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/nlpoverlap"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/nlpoverlap/>

Source (excerpt)

```
module nlpooverlap

function overLen(s text) returns number
end

function overUpper(s text) returns text
end

function overLower(s text) returns text
end

function overTrim(s text) returns text
end

function overSplit(s text, delim text) returns list text
end

function overJoin(parts list text, delim text) returns text
end

function overReplace(s text, from text, to text) returns text
end
end
```